

프로젝트 결과보고서								
(*)는 필수항목입니다.								
프로젝트 명*	SynapVox							
팀명*	SynapVox							
참가자*	구분	성명	소속	학년	구분	성명	소속	학년
	팀장	이도원	의생명융합공학부 데이터사이언스전공	3	팀원1	김도윤	의생명융합공학부 데이터사이언스전공	3
	팀원2	김용하	의생명융합공학부 데이터사이언스전공	3	팀원3	조현우	정보컴퓨터공학부 컴퓨터공학전공	3
	팀원4				팀원5			
멘토	기업 명				성명			

프로젝트 요약*	1. 프로젝트 개요 및 배경
SynapVox는 강의·회의 음성 기록을 정확히 전사하고, 시간축·의미축으로 연결된 지식 그래프로 구축해, 그 위에서 출처가 추적되는 대화를 제공하는 시스템이다. 범용 STT의 전문용어 오전사와, Graphiti를 이용하여 회의·강의가 독립 문서로 파편화되어 세션 간 연결이 끊기는 문제를 해결하고자 했다.	
2. 프로젝트 결과	
Soniox·CLOVA 관리형 STT + 2차 LLM 정제로 도메인 오전사를 완화했고, Graphiti(bi-temporal 지식 그래프 엔진)로 개념·관계·시간 이력을 자동 추출해 Neo4j에 적재했다. RAG 챗봇은 fact(관계) 단위로 검색해 [n] 인용과 함께 답변하며 근거 부족 시 기권한다. 국회 골든 데이터셋 9건(약 7시간57분) 기준 Soniox 평균 WER 23.5%(8/9 우세)를 확보했고, 49문항 골든셋 RAG 평가에서 Hit 98%, temporal(historical) hit 100%로 bi-temporal 보존을 실증했다. GitHub PR 49건으로 개발·배포(Railway+Netlify)까지 완료했다.	
※ 작성 필수: 산업체 멘토링 의견 및 반영 사항	
멘토링 피드백(2가지): ① 관리형 STT API(Soniox/CLOVA) 비용이 높아 장기적으로 부담이 된다는 지적, ② 향후 오픈소스 모델로 자체 파인튜닝하고 도메인데이터를 지금부터 수집·관리해 고도화에 활용해야 한다는 제언. 반영: 1차 서비스는 관리형 API로 운영하되, 서비스 과정에서 골든데이터셋(음성+강의자료)을 확보해향후 오픈소스 모델 파인튜닝에 사용하고, 모델만 교체해 자체 모델 체제로 전환하는 2단계 로드맵을 수립했다. 비용·보안을 함께 고려해 향후 온프레미스+클라우드 하이브리드 인프라로 전환할 계획이다. 자세한 내용은 4.4절 참고.	
개발환경*	- 개발언어 : Python (백엔드), TypeScript/JavaScript (프론트엔드) - 사용 기술 : FastAPI · Graphiti(graphiti-core, 시간 인식 지식그래프) · Neo4j Aura · Supabase(Postgres/pgvector, Auth, Storage) · Soniox/CLOVA Speech(관리형 STT) · OpenAI(gpt-5-mini/gpt-4o-mini/gpt-4o Vision/text-embedding-3-small) · React + Vite · LangSmith(관측성) - 사용 개발 Tool 및 SW, 제품의 사양 등 : GitHub(PR 기반 협업, 49건 병합) ·

	Docker(로컬 Neo4j 검증) · pytest · Railway(백엔드/그래프 서버 배포) · Netlify(프론트 배포) · Claude Code(AI 페어 프로그래밍)
--	--

< 목 차 >

1. 프로젝트 배경 및 필요성

- 1.1 문제인식 (시장 조사 및 기존 문제점 서술)
- 1.2 프로젝트의 필요성과 기대효과 (왜 이 프로젝트가 필요한지, 기대되는 효과 등)

2. 개발 목표

- 2.1 프로젝트 목표 및 세부 내용 (전체적인 개발 목표 작성, 주요 기능 및 기획 내용)
- 2.2 기존 서비스 대비 차별성 (유사 서비스 비교 및 차별점 부각)
- 2.3 사회적 가치 제공 및 도입 계획

3. 시스템 설계

- 3.1 시스템 구성도 (이미지 혹은 설명으로 시스템 아키텍처 작성)
- 3.2 사용 기술 (프론트엔드, 백엔드, API 등 구체 기술 스택)

4. 개발 결과

- 4.1 전체 시스템 흐름도 (기능 흐름 설명 및 도식화 가능)
- 4.2 기능 설명 및 주요 기능 명세서 (주요 기능에 대한 상세 설명, 각 기능의 입력/출력 및 설명)
- 4.3 디렉토리 구조
- 4.4 산업체 멘토링 의견 및 반영 사항 (멘토 피드백과 적용한 사례 정리)

5. 소개 자료 및 시연 자료(사진)

- 6.1 프로젝트 소개 자료
- 6.2 시연 사진

7. 팀 구성

- 7.1 팀원별 간단한 소개 및 역할 분담
- 7.2 팀원별 참여후기 (개별적으로 느낀 점, 협업, 기술적 어려움 극복 사례 등)

8. 참고문헌 및 출처

1. 프로젝트 배경 및 필요성

1.1 문제 인식

SynapVox는 강의·회의 음성 기록을 다루는 과정에서 실제로 확인한 두 가지 문제에서 출발했다.

문제 ① STT의 도메인 특화 부재

- 범용 STT 엔진은 일상 대화에는 잘 동작하지만, 강의·회의에 쏟아지는 전문용어에서 무너진다. 예를 들어 통계학 강의의 "프로빗", "피셔", "로지스틱회귀" 같은 영문 병기 용어가 집중적으로 오전사되어 Word Error Rate(WER)가 17~46%까지 치솟는 것을 실측으로 확인했다.
- 국회 위원회 회의 9 건(약 7 시간 57 분, 화자 1~23 인)을 국회사무처 공식 회의록과 대조한 별도 실측에서도, CLOVA Speech 평균 WER 26.9%, Soniox 평균 WER 23.5%로 — 도메인·발화 환경에 따라 관리형 API 조차 상당한 오류율을 보임을 재확인했다.
- 화자분리(diarization) 역시 두 엔진 모두 평균 절대오차 2 명 안팎으로 미흡했고, 특히 CLOVA 는 검증된 8 개 데이터셋 전부에서 검출 화자 수가 10 명을 넘지 않는 패턴이 발견되어(정확히 일치 3/8), 내부 상한 가능성이 제기되었다.

문제 ② 정리의 파편화

- 회의·강의가 각각 독립 문서로만 남으면, 세션 사이의 연결이 끊겨 무엇이 언제 다뤄졌고 어떤 주제가 이어지는지 한눈에 파악할 수 없다.
- 기존 서비스(다글로, 클로바노트, NotebookLM 등)는 단일 세션 내 요약·정리에는 강하지만, 여러 세션에 걸친 개념·결정의 연결과 시간에 따른 반복 이력 추적은 제공하지 않는다.

1.2 프로젝트의 필요성과 기대효과

(왜 이 프로젝트가 필요한지, 기대되는 효과 등)

위 두 문제를 해결하려면 (a) 도메인 오전사를 흡수·정제하는 신뢰할 수 있는 전사 파이프라인과, (b) 세션을 가로질러 개념·시간을 연결하는 지식 구조가 함께 필요하다. SynapVox는 이를 각각 STT 2-pass 파이프라인과 Graphiti 기반 시간 인식 지식그래프로 구현했다.

기대효과

- 반복되는 강의·회의 내용이 누적적으로 지식화되어, "이전에 무엇이라고 했는지" 같은 질문에 근거를 붙여 답할 수 있다.
- 결정·사실이 시간에 따라 갱신되어도(예: 강의 중 정정 공지, 회의 안건의 의결 결과 변경) 과거 시점의 상태를 정확히 복원할 수 있다 — 49 문항 골드셋 평가에서 temporal(historical) 문항 hit 100%로 실증.
- 답변마다 출처(강의·자료·세션)가 인용으로 매핑되어, 근거를 직접 확인할 수 있고 근거가 부족하면 지어내지 않고 기권한다.

2. 개발 목표

2.1 프로젝트 목표 및 세부 내용

(전체적인 개발 목표 작성, 주요 기능 및 기획 내용)

전체적인 개발 목표는 "음성 기록 → 정확한 전사 → 연결된 그래프 → 근거 기반 대화"의 4단계를 하나의 파이프라인으로 완성하는 것이었다. 주요 기능은 다음과 같다.

- STT 2-pass 파이프라인: 1 차 관리형 API(Soniox 기본/CLOVA 풀백) 전사 → 2 차 LLM(gpt-5-mini) 배치 병렬 정제
- 동일 중간포맷 스키마: Soniox·CLOVA·Whisper 세 엔진 모두 {source, segments:[{start,end,speaker,text}]]로 수렴시켜, 하위 모듈이 엔진을 몰라도 되게 함 — 엔진 교체·풀백의 기반
- Graphiti 기반 지식 그래프: 청크 텍스트를 에피소드로 투입 → LLM 이 엔티티 타입 스키마(AcademicConcept·Method·Condition·Formula)와 한국어 추출 지시문에 따라 개념·관계(fact)를 추출 → Neo4j 에 bi-temporal 로 저장
- RAG 챗봇: 질문 → 관계 fact 검색 → 근거 하이라이트 → [n] 인용이 강제된 스트리밍 답변, 근거 부족 시 기권
- 관측성: LangSmith 로 모든 OpenAI 호출(답변 생성·개념 추출·임베딩·재랭킹)의 토큰·비용·지연시간·재시도 횟수를 프로젝트/회의 단위로 추적

2.2 기존 서비스 대비 차별성

(유사 서비스 비교 및 차별점 부각)

비교 축	다글로 · 클로바노트 · NotebookLM 등	SynapVox
세션 간 연결	단일 세션 요약에 그침	시간축·의미축으로 세션을 가로질러 연결
시간에 따른 사실 변화	최신 상태만 반영(덮어쓰기)	bi-temporal — 과거 상태를 정확히 복원 (historical hit 100%)
답변의 근거 추적	출처 매핑 없음 또는 문서 단위	관계(fact) 단위 검색 + [n] 인용을 실제 강의/자료로 매핑
신뢰성 장치	일반적인 요약/답변	근거 부족 시 기권(abstain) — 할루시네이션 억제

※ 위 비교는 팀 자체 경쟁사 분석(다글로/클로바노트/NotebookLM 공개 기능 기준) 및 본 프로젝트 실측 결과를 근거로 한다.

2.3 사회적 가치 제공 및 도입 계획

(프로젝트의 공공성, 지속가능성, 환경보호 등)

- 강의 복습·회의록 자동화를 통해 학습자·직장인의 반복 정리 부담을 줄이고, 근거 기반 대화로 잘못된 정보의 확산(할루시네이션)을 억제하는 방향으로 설계했다.
- 도입 계획: 현재 Railway(백엔드/그래프 서버) + Netlify(프론트엔드) 배포로 실사용 가능한 프로토타입을 확보했다. 향후 (1) 도메인 어휘 부스팅(CLOVA boostings/Whisper initial_prompt 실적용), (2) 도메인 데이터셋 확보 후 자체 호스팅 STT 파인튜닝, (3) 오전사 의심 구간만 선별 정제하는 방식으로 성능과 비용을 함께 개선할 계획이다.

3. 시스템 설계

3.1 시스템 구성도

(이미지 혹은 설명으로 시스템 아키텍처 작성)

입력(음성 녹음 + 강의.회의 자료) → STT 파이프라인(1차 전사 + 2차 LLM 정제) → chunking → 지식 그래프 적재(Graphiti → Neo4j) → 두 가지 소비 경로(그래프 뷰 / RAG 챗봇) 순서로 구성된다.

- STT 파이프라인: 음성.자료 입력 → Soniox(기본)/CLOVA(폴백) 전사 → gpt-5-mini 2 차 정제 → 중간포맷 JSON 출력
- Chunking: 중간포맷 세그먼트를 화자 발화 형태의 평문으로 재구성해 에피소드 단위로 분할
- 지식 그래프 적재: Graphiti(graphiti-core)가 각 에피소드에서 개념(Entity)-관계(fact)를 추출하고 시간축(bi-temporal)을 보존해 Neo4j 에 저장
- 소비: (a) 그래프 뷰 — session/concept 노드를 프론트엔드에 렌더링, (b) RAG 챗봇 — 관계 fact 를 검색해 인용 기반 답변 생성

핵심 설계 포인트: 세 STT 엔진(Soniox-CLOVA-Whisper)이 모두 동일한 중간포맷 스키마로 수렴하므로, 하위 모듈(정제.그래프 적재)은 어떤 엔진이 전사했는지 몰라도 되도록 설계했다 — 이것이 엔진 교체(A/B 비교)와 장애 시 자동 폴백을 가능하게 하는 기반이다.

3.2 사용 기술

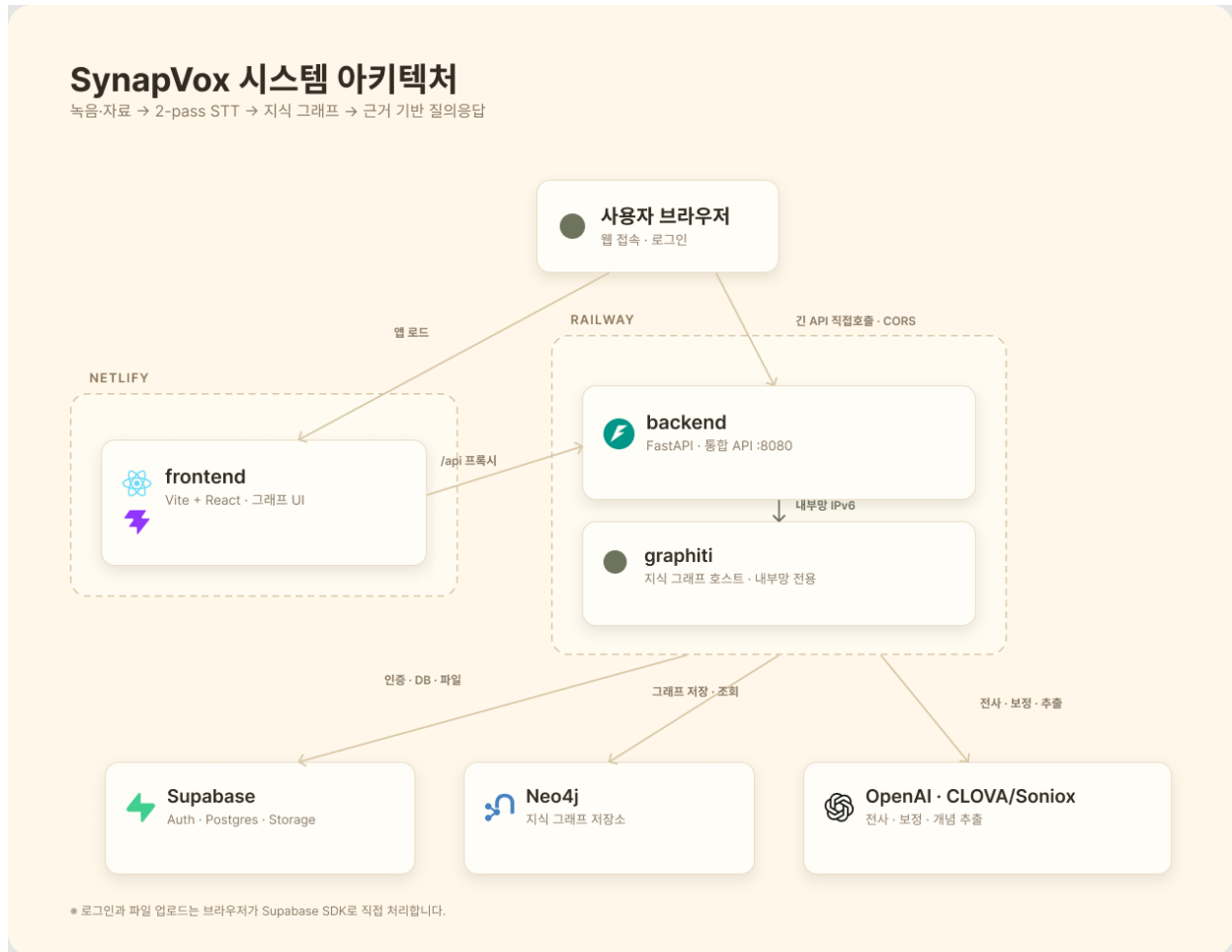
(프론트엔드, 백엔드, API 등 구체 기술 스택)

계층	기술	역할
STT	Soniox(기본) / CLOVA Speech(폴백)	음성 → 텍스트 + 화자분리 (공통 디스패처 stt.py)
STT 2차 정제	gpt-5-mini (reasoning_effort=low)	오전사 교정 · 문맥 보정, 배치 병렬
자료 이미지 설명	gpt-4o (Vision)	PDF·PPTX 임베딩 이미지 텍스트화
지식 그래프	Graphiti(graphiti-core) + Neo4j Aura	개념.관계 추출, bi-temporal 저장
임베딩	text-embedding-3-small	개념.청크 벡터화
보조 DB	Supabase (Postgres/pgvector, Auth, Storage)	회의자료.전사문 저장, 인증, 자료 파일 보관
챗봇 답변	gpt-4o-mini	RAG 답변 생성 (근거 기반, 스트리밍)
관측성	LangSmith	토큰.비용.지연시간.재시도 추적
백엔드	Python, FastAPI	API 서버, STT/그래프 오케스트레이션
프론트엔드	React, Vite, TypeScript	그래프 뷰, RAG 챗봇 UI
배포	Railway(백엔드/그래프 서버), Netlify(프론트)	실사용 가능한 프로토타입 호스팅

4. 개발 결과

4.1 전체 시스템 흐름도

(기능 흐름 설명 및 도식화 가능)



- ① 사용자가 녹음 파일(또는 실시간 녹음) + 강의·회의 자료를 업로드
- ② STT 1차 전사(Soniox 기본, 실패 시 CLOVA 폴백) — 내용 해시로 동일 파일 재업로드 시 중복 적재 방지
- ③ STT 2차 정제 — 배치 병렬 LLM 호출로 오전사 교정, 세그먼트 ID 검증 실패 시 최대 재시도, 일부 배치 실패해도 해당 조각만 원문 유지(전체 재호출 없음)
- ④ 자료(자료 이미지 포함) 텍스트 추출 → chunking → Graphiti 에피소드로 적재(그래프 중복 적재 방지 — 소스 재적재 시 409)
- ⑤ Graphiti 가 개념·관계·시간축을 추출해 Neo4j 에 저장 (project 별 group_id 네임스페이스, 회의 단위 meeting_id 스코프 검색 지원)
- ⑥ 그래프 뷰 렌더링 + RAG 챗봇 — 질문 시 관계 fact 검색 → 근거 하이라이트 → [n] 인용이 포함된 답변을 토큰 단위로 실시간 스트리밍

4.2 기능 설명 및 주요 기능 명세서

(주요 기능에 대한 상세 설명, 각 기능의 입력/출력 및 설명)

기능	입력	출력	설명
STT 전사	오디오 파일	중간포맷 JSON (화자분리 포함)	Soniox 기본/CLOVA 풀백 디스패처, 내용 해시 기반 중복 적재 방지
STT 2차 정제	1차 전사 + 강의자료(선택)	정제된 전사문	배치 병렬 LLM 호출, 세그먼트ID 검증 + 재시도, 부분 실패 격리
자료 텍스트 추출	PDF/PPTX 파일	본문 텍스트 (+이미지 설명)	임베딩 이미지는 gpt-4o Vision으로 텍스트화
그래프 적재	청크 텍스트 + 날짜	Neo4j 노드/엣지	Graphiti add_episode, reference_time으로 그래프 시간축 반영
RAG 챗봇 답변	사용자 질문	인용 포함 스트리밍 답변	관계 fact 검색 → 근거 하이라이트 → [n] 인용 강제, 근거 부족 시 기권
그래프 뷰	프로젝트 ID	노드/엣지 시각화	session(Episodic)/concept(Entity) 노드 렌더링
회의 단위 검색 스코프	질문 + meeting_id	해당 회의로 좁혀진 답변	group_id는 프로젝트 단위 유지(dedup 보존), 에피소드 제목 매칭으로 사후 필터링
관리자 품질 대시보드	-	STT 평가 이력 카테고리별 정리	실데이터 연결, 관측성 지표 확인 현재는 LangSmith로 관리

4.3 디렉토리 구조

```
synapvox/  
├── backend/  
│   ├── stt/                # STT 1·2차 파이프라인, stt.py 디스패처 (Soniox 기본/CLOVA 풀백)  
│   ├── graphrag/          # pgvector VectorStore, HybridSearch (baseline 검색 경로)  
│   └── integration/        # FastAPI 서버 (api/main.py), graphiti_host.py,  
gsvx_connector.py  
├── frontend/               # React + Vite + TypeScript (그래프 뷰, RAG 챗봇 UI)  
├── services/graphiti/      # graphiti-core, graph_service (git submodule, Zep 오픈소스)  
├── supabase/migrations/    # Postgres 스키마 마이그레이션 (project_sources, chat_sessions 등)  
├── schemas/                # 모듈 간 계약 스키마 (중간포맷 JSON 등)  
└── docs/                   # 아키텍처/의사결정 문서 (ADR 등)
```

4.4 산업체 멘토링 의견 및 반영 사항

(멘토 피드백과 적용한 사례 정리)

멘토링 의견 (2가지)

- ① API 비용 문제: 현재 파이프라인이 Soniox/CLOVA 같은 관리형 STT API에 의존하고 있어, 서비스 규모가 커질수록 호출당 과금이 누적되어 장기적으로 비용 부담이 크다는 지적을 받았다.
- ② 데이터 확보 및 자체 모델화 필요성: 장기적으로는 오픈소스 모델을 자체 파인튜닝해 API 의존도를 낮춰야 하며, 이를 위해 지금부터 도메인 데이터(음성+강의자료)를 체계적으로 수집·관리해 향후 고도화에 활용해야 한다는 피드백을 받았다.

반영 사항

- 골든데이터셋 확보: 서비스 운영 과정에서 쌓이는 음성 데이터와 강의회의 자료를 체계적으로 수집해, 향후 파인튜닝용 골든데이터셋으로 활용할 계획이다.
- 하이브리드 인프라 전환: 비용과 보안을 동시에 고려해, 향후 온프레미스(자체 서버)와 클라우드를 병행하는 하이브리드 구조로 전환할 계획이다.
- 단계적 파인튜닝 로드맵(거시적 계획): 1차 서비스는 지금처럼 관리형 API(Soniox 기본/CLOVA 풀백)를 통해 운영하며 그 사이 골든데이터셋을 축적한다. 충분한 데이터가 모이면 오픈소스 STT 모델을 도메인 데이터로 파인튜닝해 API 호출을 자체 모델로 대체한다. 이때 모델 계층만 교체되고 중간포맷 스키마 등 나머지 파이프라인은 그대로 유지되도록 이미 설계돼 있어, 전환 시점의 리스크를 최소화할 수 있다.

5. 소개 자료 및 시연 영상

5.1 프로젝트 소개 자료

발표자료(SynapVox_발표.pptx, 총 15장)와 발표대본(SynapVox_발표대본.docx)을 별도 파일로 준비했다. 구성은 다음과 같다.

- 문제 정의(2 장): STT 도메인 특화 부재, 정리의 파편화
- 시스템 아키텍처(1 장): 입력→STT→그래프→소비 전체 흐름
- 구현 ①STT ②지식그래프 ③RAG 챗봇(5 장): 각 구현의 설계 근거와 세부 동작
- 사용 모델·관측성(2 장): 단계별 LLM 모델 선택 근거, LangSmith 토큰 추적
- 성능 평가(4 장): 평가 설계, temporal 검증, STT 결과, RAG 결과
- 한계와 발전 방향, 요약(2 장)

5.2 시연 영상

(영상 링크 또는 주요 장면 설명)

<https://youtu.be/rMqPj1id52c>

6. 팀 구성

6.1 팀원별 소개 및 역할 분담

이름	역할	담당 영역
이도원 (팀장)	PM · 통합 · 프론트엔드	E2E 통합(api/main.py), 인증(Supabase Auth), 그래프 뷰·채팅 UI, 배포(Railway/Netlify), 일정 관리
김도윤	LLM/프롬프트	LLM 출력 스키마, 프롬프트 설계, 검증·재시도 루프, RAG 평가셋 설계
김용하	백엔드 · DB · 그래프	Graph/Vector DB 스키마, Neo4j 적재, 하이브리드 검색, 그래프 인프라
조현우	데이터 파이프라인 · 전처리	STT 엔진 선정·연동(Soniox/CLOVA), 화자분리 검증, 자료 텍스트 추출, 중간포맷 JSON 변환기

6.2 팀원별 참여후기

(개별적으로 느낀 점, 협업, 기술적 어려움 극복 사례 등)

이도원 (팀장 · PM · 통합)

이번 프로젝트에서 저는 팀장으로서 PM과 통합을 맡았는데, 솔직히 이 부분이 가장 힘들었습니다. 도윤·용하·현우 세 사람이 각자 LLM, 그래프 DB, STT 파이프라인을 병렬로 개발하다 보니, 제 역할은 그 세 갈래를 하나의 서비스로 묶어내는 것이었는데 스키마 하나가 어긋나면 그게 전체 파이프라인 끝까지 전파된다는 걸 뼈저리게 느꼈습니다. 특히 그래프 엔진을 자체 구현(graphrag)에서 Graphiti 기반 자체 호스팅으로 옮기는 결정을 내부적으로 몇 번이나 재검토하면서, 다른 팀원들이 이미 쌓아둔 코드와 계속 어긋나는 부분을 그때그때 다시 맞추는 게 체력적으로도 정신적으로도 가장 부담스러운 일이었습니다. 배포 단계에서도 Railway/Netlify 설정, launch.json 경로 오류, .env 자동 로드 누락처럼 사소해 보이지만 서비스 전체를 멈추는 문제들을 계속 발견하고 고쳐야 해서, PM이라는 역할이 “다 되어 있는 걸 확인만 하는 자리”가 아니라 “안 되는 걸 제일 먼저 알아채고 책임지는 자리”라는 걸 절실히 배웠습니다. 그래도 네 명이 만든 조각들이 실제로 하나의 서비스로 붙어서 동작하는 걸 볼 때마다, 이 고생이 헛되지 않았다는 확신이 들었습니다.

김도윤 (LLM · 프롬프트)

저는 LLM 출력 스키마와 RAG 평가셋 설계를 맡았는데, 가장 어려웠던 건 골드셋 49문항을 사람이 검수할 수 있는 수준으로 다듬는 과정이었습니다. 특히 temporal(historical) 문항은 강의 중 정정된 내용을 과거 시점 기준으로 물어야 진짜 bi-temporal 검증이 되는 거라, 문항 하나하나가 세션 번호나 대본 인용에 기대지 않고 개념만으로답을 가려낼 수 있는지 몇 번이고 다시 설계해야 했습니다. 환각 주장 비율이 24~26%로 나왔을 때는 저희 프롬프트 설계의 어디가 문제인지 계속 의심하게 됐는데, 인용 정확도와 충실도를 나눠서 보니 문제의 상당 부분이 STT 오전사로 오염된 fact 쪽에 있다는 걸 알게 되어 오히려 다음 방향이 명확해졌습니다. 팀원들이 만든 그래프·전사 결과를 숫자로 평가하는 역할이다 보니 제 판단 하나로 팀 전체의 성과가 좋아 보이기도 나빠 보이기도 한다는 부담이 컸지만, 그만큼 더 엄격하게 설계하려고 노력했습니다.

용하 (백엔드 · DB · 그래프)

저는 그래프/벡터 DB 스키마와 검색 인프라를 맡았는데, 가장 큰 고비는 Microsoft graphrag로 시작했다가 Graphiti로, 다시 자체 호스팅 Graphiti로 아키텍처를 두 번이나 갈아탄 것이었습니다. 매번 팀원들이 이미 그 위에 쌓아둔 코드가 있었기 때문에, 결정을 내릴 때마다 “이번엔 정말 마지막이어야 한다”는 압박이 있었습니다. pgvector 테이블의 PRIMARY KEY를 제거 고쳤다가 다른 작업 중에 실수로 되돌아간 걸 나중에야 알게 됐을 때는, 스키마 계약이 얼마나 쉽게 깨질 수 있는지, 그리고 그걸 막으려면 테스트와 문서화가 얼마나 중요한지 다시 느꼈습니다. group_id를 프로젝트 단위로 유지할지 미팅 단위로 쪼갤지 고민할 때도, 눈앞의 기능(미팅별 검색)만 보면 쪼개는 게 맞아 보이지만 Graphiti의 dedup 로직까지 이해해야 진짜 트레이드오프가 보인다는 걸 깨달았던 게 기억에 남습니다.

조현우 (데이터 파이프라인 · 전처리)

저는 STT 엔진 선정과 데이터 파이프라인을 맡았는데, 로컬 Whisper+pyannote로 시작했다가 ADR-005 때문에 CLOVA로, 나중엔 Soniox까지 STT 경로를 세 번이나 바뀌가며 실측했습니다. 국회 회의록 9건, 약 8시간 분량 데이터셋으로 WER-CER-화자분리를 전부 실제로 재고 나서야 “느낌상 낫다”가 아니라 숫자로 Soniox를 선택할 수 있었던 게 이번 프로젝트에서 가장 값진 경험이었습니다. 특히 CLOVA가 검증된 8개 데이터셋 전부에서 화자 수를 10명 넘게 검출한 적이 없다는 패턴을 확인했을 때는, 우연이 아니라 진짜 시스템의 한계를 발견했다는 확신이 들어서 짜릿했습니다. 환경 설정 과정에서도 pgvector 스키마가 되돌아간 버그를 git blame으로 추적하거나, 로컬 Neo4j를 직접 띄워 실제 인프라 전체로 서비스를 검증하는 등, 코드를 짜는 것 못지않게 “이게 진짜 되는지 실측으로 확인하는 습관”이 왜 중요한지 몸으로 배운 프로젝트였습니다.

7. 참고문헌 및 출처

데이터 출처

- 국회 영상회의록시스템(w3.assembly.go.kr)— STT 성능 비교용 오디오 원본
- 국회사무처 공식 회의록(record.assembly.go.kr) — WER/CER/화자분리 정답 텍스트
- 「국회법」 제149조제2항상업적 목적 사용 금지, 본 프로젝트는 비상업적 내부 검증 용도로만 사용

기술 문서 및 라이브러리

- Graphiti (graphiti-core, Zep) —시간 인식 지식그래프 엔진, <https://github.com/getzep/graphiti>
- Neo4j Aura — 그래프 데이터베이스 호스팅
- Supabase — Postgres/pgvector, Auth, Storage
- Soniox API, NAVER CloudPlatform CLOVA Speech API — 관리형 음성 인식
- OpenAI API (gpt-5-mini, gpt-4o-mini, gpt-4o Vision, text-embedding-3-small)
- LangSmith — LLM 호출 관측성(토큰·비용·지연시간 추적)

내부 산출물

- SynapVox STT 파이프라인 검증 분석보고서(2026-07-16 작성, 2026-07-19 CER 분석 추가) — 국회 골든 데이터셋9건 기준CLOVA vs Soniox 실측 비교
- SynapVox GraphRAG 평가 보고서v2 (2026-07-19, 커밋7fcfb4b) — 골드셋49문항 기준 텍스트 경로/STT 경로RAG 품질 평가
- GitHub 저장소synapvox/synapvox — Pull Request #1~#49 (STT 파이프라인, pgvector/Neo4j 스키마, Graphiti 통합, 인증, 배포 등 전 이력 관리)
- ADR-005 (내부 아키텍처 결정 기록) — 화자분리 내장 관리형STT API 채택 근거